



中华人民共和国通信行业标准

XX/T s—XXXX

测试智能体能力成熟度模型

Test the capability maturity model of intelligent agents

[点击此处添加与国际标准一致性程度的标识]

(征求意见稿)

2025 年 10 月 16 日

XXXX - - XX 发布

XXXX - XX - 实施

中华人民共和国工业和信息化部 发布

目次

前 言 II

测试智能体能力成熟度模型 1

1. 范围 1

2. 规范性引用文件 1

3. 术语和定义 1

3.1 1

4. 缩略语 1

5. 测试智能体能力成熟度模型架构 1

6. 测试智能体能力成熟度模型分级规则 2

6.1. 分级概述 2

6.2. 分级规则 3

7. 智能体场景应用 3

7.1. 测试设计 3

7.2. 测试执行 6

7.3. 测试分析 9

8. 智能体工程能力（认证可选模块） 12

8.1. 自然语言理解 12

8.2. DOM 智能分析 12

8.3. 上下文工程 13

8.4. 测试数据服务 13

8.5. 元素定位 14

8.6. API/SDK 15

8.7. 测试管理 15

8.8. DevOps 16

8.9. 可观测 16

9. 智能体构建 18

9.1. 模型配置 18

9.2. 提示词 19

9.3. 流程编排 19

9.4. 效果验证 20

9.5. 知识库 21

前 言

本文件按照GB/T 1.1—2020《》给出的规则起草。

请注意本文件的某些内容可能涉及专利。本文件的发布机构不承担识别这些专利的责任。

本文件由中国通信标准化协会提出并归口。

本文件起草单位：xxx

本文件主要起草人：xxx、xxx等人

测试智能体能力成熟度模型

1. 范围

本标准构建了测试智能体能力成熟度模型分级目标，覆盖智能体场景应用、智能体工程能力、智能体构建三个核心板块测试智能体领域能力要求。

本标准适用于各行企业对其组织或项目的测试能力成熟度评估，包括软件测试智能体的提供方，以及软件智能化测试体系建设方。针对不同使用场景，本文件对其技术指标的要求略有不同。

2. 规范性引用文件

下列文件对于本文件的应用是必不可少的。凡是注日期的引用文件，仅所注日期的版本适用于本文件。凡是不注日期的引用文件，其最新版本（包括所有的修改单）适用于本文件。

3. 术语和定义

下列术语及定义适用于本文件

3.1

测试智能体 AI Testing Agent

模拟人类测试工程师的行为，将 AI 能力应用于软件测试领域，能够自主或半自主地完成测试设计、执行、分析等任务且具备自主感知、分析、决策和执行能力的智能实体。

4. 缩略语

下列缩略语适用于本文件：

CI：持续集成（Continuous Integration）

CD：持续交付/持续部署（Continuous Delivery/Continuous Deployment）

API：应用程序编程接口（Application Programming Interface）

5. 测试智能体能力成熟度模型架构

本文件所定义的测试智能体能力成熟度模型架构，如图1所示：

智能体场景应用层：定义了智能体能够承担的具体测试任务角色，直接面向最终用户的业务场景，封装了端到端的测试活动。

智能体工程能力层：连接上层应用与底层平台的枢纽。赋予智能体规划测试任务、操作测试工具及管理测试上下文的核心行为能力。

智能体构建层：集成测试领域知识、资产与专用模型，为智能体提供精准、稳定的数据支撑与流程编排能力，是智能测试的基石。



图 1 测试智能体能力成熟度模型架构

6. 测试智能体能力成熟度模型分级规则

6.1. 分级概述

测试智能体能力成熟度模型，共分为3个级别，每个级别中按照不同程度说明。各级别对应的具体能力要求见第7-9章。各级别技术要求描述如下：

—基础级：智能体具备基础的任务执行能力，可基于预设提示词完成特定、孤立的测试任务，如脚本生成或结果检查。其能力依赖明确指令，自动化水平有限，缺乏深度分析与上下文感知。

—增强级：智能体能系统化整合测试流程，如将案例设计、执行与分析串联。具备良好的上下文理解与数据服务能力，可进行跨环节的精准测试与风险分析，实现一定程度的自主优化。

—先进级：智能体具备前瞻性预测与自主决策能力。通过知识图谱与深度复盘，能主动识别潜在性能风险、精准诊断复杂研发问题，并持续自我演进，驱动测试过程的全面智能化与优化。

6.2. 分级规则

测试智能体能力成熟度模型，能力要求项分为1级、2级、3级。

满足90%及以上的1级能力项，即可认为达到基础级要求；

满足90%及以上的1级能力项且满足90%及以上的2级能力项，即可认为达到增强级要求；

满足90%及以上的1级能力项、90%及以上的2级能力项且满足90%及以上的3级能力项，即可认为达到先进级要求。

可选项供参考，不作为必须满足项。

7. 智能体场景应用

7.1. 测试设计

7.1.1. 测试需求分析与方案生成

1 级：

- 1) 应支持读取和理解结构化的需求文档（如用户故事、需求规格说明书），提取关键测试点；
- 2) 应支持识别需求描述中的动作、操作对象、输入数据和预期结果；
- 3) 应支持根据提供的测试方案模板生成测试方案。

2 级：

- 1) 应支持理解业务规则和流程，能够分析需求中不同功能点之间的关联；
- 2) 应支持识别需求变更可能引发的直接风险；
- 3) 应支持识别需求文档中术语不一致或明显存在的逻辑矛盾。

3 级：

- 1) 应支持结合领域知识库，理解系统架构和技术栈，能够设计针对架构弱点的测试（如分布式系统的网络分区、数据库故障等）；
- 2) 应支持根据需求生成业务模型（如状态图、流程图）；
- 3) 应支持从历史缺陷中学习，识别常见缺陷模式，并在测试需求分析中优先考虑这些模式；
- 4) 应支持根据项目上下文（如项目阶段、风险等级、资源约束）制定测试策略，确定测试重点和测试方法；
- 5) 应支持深入分析变更对系统架构、历史功能的连带影响，并自动确定回归测试范围。

7.1.2. 测试案例设计

1 级：

- 1) 应支持根据明确的结构化需求文档生成基础功能测试案例，覆盖核心业务流程；
- 2) 应支持提供测试案例的基本属性编辑功能；
- 3) 应支持测试案例的简单查询与导出功能，支持 Excel、Word 等常用格式；
- 4) 应支持识别需求文档中的明显矛盾点，以及提供测试用例基本要素的完整性检查，包括前置条件、操作步骤、预期结果等。

2 级：

- 1) 应支持基于需求关联分析，生成多场景组合测试案例，包括正常场景、异常场景、边界场景；
- 2) 应提供测试案例的优先级自动分级功能，基于业务重要性、风险等级、用户使用频率等维度进行判定；
- 3) 应支持测试案例与需求的双向追溯，清晰记录每个测试案例对应的需求点及追溯关系；
- 4) 应支持自定义测试案例模板，适配不同项目类型；
- 5) 应支持基于需求变更的测试用例增量生成与更新。

3 级：

- 1) 应支持基于历史缺陷分布，动态优化测试案例集，提升高风险区域覆盖度；
- 2) 应支持根据需求生成业务模型（如状态图、流程图），并基于模型自动生成测试用例；
- 3) 应支持与主流需求管理系统集成，实现需求变更时的测试案例自动更新与同步；
- 4) 宜支持基于用户行为数据的真实场景用例生成；
- 5) 宜能覆盖复杂业务交互场景。

7.1.3. 风险分析

1 级：

- 1) 应支持识别需求文档中明确标注的高风险功能点；
- 2) 应支持基于关键词匹配，识别常见风险类型（如性能、安全、兼容性等）；
- 3) 应支持对测试范围中已识别风险的简单记录与分类；
- 4) 应支持标记测试案例中与已知风险关联的测试点。

2 级：

- 1) 应支持结合业务逻辑与系统架构，识别潜在的业务流程中断点，以及数据一致性风险；
- 2) 应支持基于历史缺陷数据，识别高频故障模块或功能，并关联至测试重点；
- 3) 应支持分析需求变更对现有功能的影响，识别回归测试中的风险范围；
- 4) 应支持多维度风险评级（如发生概率、影响程度），并给出测试优先级建议；
- 5) 应支持识别测试数据、测试环境等非功能因素可能引致的测试执行风险；

- 6) 应支持生成风险跟踪矩阵，关联风险点与对应测试案例；
- 7) 应支持风险等级划分，并给出对应风险等级的应对建议；
- 8) 应支持生成结构化的风险分析报告，包含风险点、等级、影响范围、应对措施等内容。

3 级：

- 1) 应支持基于系统架构、代码变更、运行监控等数据，预测潜在的技术风险点；
- 2) 应支持利用机器学习模型，结合项目上下文与历史数据，动态评估与调整风险等级；
- 3) 应支持构建风险传播模型，分析风险点对上下游功能模块的连带影响；
- 4) 应支持智能推荐风险缓解策略与针对性测试加固方案；
- 5) 宜具备风险感知的自适应能力，根据测试执行过程中的变化点动态更新风险视图；

7.1.4. 测试案例检查

1 级：

- 1) 应支持检查测试案例的基本格式规范性；
- 2) 应支持识别测试案例中的明显逻辑错误；
- 3) 应支持检查测试案例的编号唯一性，避免编号重复；
- 4) 应实现重复或高度相似测试案例的基础检测；
- 5) 应支持提示测试案例中缺失的关键属性。

2 级：

- 1) 应支持检查测试案例与需求的一致性，识别与需求描述不符的案例内容；
- 2) 应支持分析测试案例的可执行性，提示执行过程中可能存在的障碍；
- 3) 应支持检查测试案例的冗余性，识别对测试目标无贡献的无效案例；
- 4) 应支持结合项目测试标准，自定义案例检查规则；
- 5) 应支持批量案例检查，并生成检查报告，明确问题案例及整改建议；
- 6) 应支持跟踪问题案例的整改情况，支持整改后的二次校验。

3 级：

- 1) 应支持测试案例的深度评审，识别潜在的隐性问题；
- 2) 应支持智能识别测试案例集的业务逻辑盲区与覆盖缺口；
- 3) 应支持结合历史缺陷数据，分析测试案例的缺陷发现能力，优化低效能案例；
- 4) 应支持与测试执行结果联动，检查未发现缺陷的案例是否存在设计缺陷；
- 5) 应提供测试案例的自适应优化建议，根据线上反馈和生产缺陷动态调整案例库的优先级和内容；

- 6) 宜建立案例质量评分体系, 基于检查结果对案例质量进行量化评估。

7.1.5. 自动化脚本生成

1 级:

- 1) 应支持常见测试框架的脚本模板生成;
- 2) 应支持脚本基本语法检查与错误提示。

2 级:

- 1) 应支持基于自然语言描述生成可执行测试脚本;
- 2) 应支持脚本参数化配置, 实现数据驱动测试;
- 3) 应支持多端脚本自动适配, 如 Web、移动端;
- 4) 应支持脚本执行失败的智能恢复机制。

3 级:

- 1) 应支持跨应用的端到端流程脚本自动编排;
- 2) 应能识别脚本执行瓶颈, 支持脚本性能优化建议生成;
- 3) 宜支持基于历史执行数据的脚本稳定性预测;
- 4) 宜支持脚本代码质量评估, 提供重构优化建议。

7.2. 测试执行

7.2.1. 智能脚本质量分析

1 级:

- 1) 应支持对自动化脚本的语法正确性进行检查, 识别语法错误;
- 2) 应支持分析脚本的基础可执行性, 提示缺失的依赖组件或配置;
- 3) 应支持检查脚本中明显的硬编码问题;
- 4) 应实现脚本与测试案例步骤的基础一致性比对。

2 级:

- 1) 应支持分析脚本的稳定性, 识别易受环境变化影响的脚本片段;
- 2) 应支持检查脚本的复用性, 提示可封装的通用功能模块;
- 3) 应支持分析脚本的执行效率, 识别冗余的代码逻辑或重复操作;
- 4) 应支持检查脚本的错误处理机制完整性, 提示缺失的异常捕获代码;
- 5) 应支持自定义脚本质量检查规则, 适配不同项目的编码规范;
- 6) 应支持生成脚本质量分析报告, 包含问题清单、严重程度及优化建议。

3 级:

- 1) 应支持预测脚本在不同测试环境下的兼容性风险；
- 2) 应支持自动优化脚本质量问题；
- 3) 应支持与脚本执行日志联动，分析高频失败脚本的根本原因并给出整改方案；
- 4) 宜支持建立脚本质量评估模型，结合执行稳定性、复用率、效率等指标进行综合评分；
- 5) 宜支持根据测试需求变化，自动调整脚本质量检查重点。

7.2.2. 精准测试

1 级：

- 1) 应支持基于代码变更范围，精准识别受变更影响的测试用例集，排除无关用例，实现定向回归测试，避免全量回归带来的资源浪费与效率损耗。
- 2) 应支持建立测试用例与代码模块的关联映射关系，明确每个用例对应的具体代码覆盖范围，为后续精准定位问题提供基础支撑。
- 3) 应支持测试覆盖度的基础统计功能，可统计代码行覆盖、函数覆盖等核心指标，直观呈现测试覆盖情况，辅助判断测试是否充分。
- 4) 应支持根据预设的优先级规则，从测试用例集中筛选出核心业务模块、高风险功能相关的高优先级用例，优先安排执行，保障关键功能质量。

2 级：

- 1) 应支持结合历史缺陷数据，分析各代码模块的缺陷密度，识别出缺陷频发的高风险代码模块，针对性加大该区域的测试用例投入与执行频次。
- 2) 应支持基于接口调用关系图谱，追溯代码变更可能影响的间接关联模块，扩展回归测试覆盖边界，避免因关联模块未测试导致缺陷遗漏。
- 3) 应支持测试覆盖度的多维度分析，除基础指标外，还可统计分支覆盖、条件覆盖等精细指标，并自动识别覆盖缺口，提示补充测试用例。
- 4) 应支持根据测试执行结果，动态调整用例优先级，将发现缺陷概率高的用例优先级提升，优先执行以提高缺陷发现效率，缩短缺陷暴露时间。
- 5) 应支持与代码管理工具深度集成，自动捕获代码提交信息与变更内容，无需人工干预即可触发针对变更内容的针对性测试，减少人工操作成本。

3 级：

- 1) 应支持基于机器学习算法，分析历史代码变更与缺陷数据，构建缺陷预测模型，精准预测当前代码变更可能引入缺陷的概率，为测试资源精准分配提供决策支持。
- 2) 应支持结合用户行为数据，分析用户高频操作路径与业务场景权重，优先覆盖高频高价值用

户路径对应的测试场景，确保用户核心体验不受影响。

- 3) 应支持实时监控测试覆盖进度，对比预设的覆盖目标，当发现覆盖缺口时，自动分析缺口类型并补充适配的测试用例，实现覆盖度动态达标。
- 4) 宜支持跨模块依赖的深度分析，通过梳理模块间的隐性依赖关系，识别代码变更可能影响的隐性路径，并将其纳入测试范围，避免隐性缺陷遗漏。
- 5) 宜支持测试资源的智能调度功能，实时监控算力、人力等资源占用情况，结合测试任务优先级，动态分配资源，确保高优先级任务高效执行。

7.2.3. 测试结果检查

1 级：

- 1) 应支持基于预定义断言规则，自动化判定测试结果；
- 2) 应支持聚合多场景测试执行结果，以图表等形式可视化呈现；
- 3) 应支持整理测试信息，包括自动捕获错误日志、堆栈跟踪、截图等测试信息。

2 级：

- 1) 应支持结合多源数据，分析测试失败根因；
- 2) 应支持智能识别共性缺陷，实现缺陷聚合去重；
- 3) 应支持定量分析测试成效（测试用例有效性，测试效率、缺陷率、覆盖率等）。

3 级：

- 1) 应支持根据预测结果与当前测试数据，动态调整测试策略；
- 2) 应支持基于历史测试结果、变更频率等数据，进行预测性分析。

7.2.4. 报告自动生成

1 级：

- 1) 应支持按预设模板，自动汇总总用例数、通过/失败数、执行时间等基础数据；
- 2) 应支持生成固定格式的结构化报告文档；
- 3) 应支持聚合多轮测试、多测试套件结果，通过基础图表可视化展示；
- 4) 应支持按模块、测试类型等维度对测试结果进行初步分类统计；
- 5) 应支持报告导出功能，兼容 Excel、PDF 等常用文件格式；
- 6) 应支持错误日志、堆栈信息、截图等执行记录与失败用例关联展示；
- 7) 支持关联附件归档存储，并在报告中提供查阅入口。

2 级:

- 1) 应支持从模块、需求、缺陷严重性等多维度聚合测试结果;
- 2) 应支持失败用例与代码变更、相关缺陷、系统监控事件关联展示;
- 3) 应支持通过图表钻取、筛选等功能深入探索测试数据;
- 4) 应支持不同测试周期的结果数据对比能力, 识别质量趋势和回归问题;
- 5) 应支持自动生成测试结论, 包括质量评估、风险提示和改进建议。

3 级:

- 1) 应支持预测缺陷修复后的质量趋势, 为版本决策提供数据支撑;
- 2) 应支持自动生成定制化报告, 提供针对性的决策信息。如支持根据不同角色需求生成个性化报告;
- 3) 应支持基于历史测试、代码变更等数据, 预测高风险模块;
- 4) 应支持对预测风险进行量化提示, 纳入质量风险报告;
- 5) 应支持关键结论自动触发下游测试、工单分派等行动, 构建报告驱动的测试质量自动化闭环。

7.3. 测试分析

7.3.1. 测试过程复盘

1 级:

- 1) 应支持自动汇总测试执行的基础数据, 如测试用例总数、通过率、缺陷数量等;
- 2) 应支持按模块、测试轮次等维度对测试结果进行初步的分类统计;
- 3) 应支持当前版本与历史基线的测试指标自动化对比;
- 4) 应支持基于日志梳理关键阶段时间节点与事件, 生成复盘时间线。

2 级:

- 1) 应支持从需求覆盖率、测试用例覆盖率、缺陷逃逸率、测试周期等多维度度量, 自动化分析效率瓶颈与质量短板;
- 2) 应支持分析缺陷的引入阶段和发现阶段, 初步评估开发过程和测试过程的有效性;
- 3) 应支持测试缺陷与相关代码、配置等关联, 定位根本原因;
- 4) 应支持基于分析结果, 自动生成核心结论, 问题根因总结等信息。

3 级:

- 1) 应支持基于历史复盘数据, 识别测试策略中重复低效模块与质量风险趋势;

- 2) 应支持预测下一测试周期可能出现的问题和风险；
- 3) 应支持自动生成可执行改进任务，并跟踪落地情况与效果验证；
- 4) 应支持复盘结论中包含测试策略优化建议、资源优化分配等。

7.3.2. 缺陷根因分析

1 级：

- 1) 应支持测试失败结果与原始证据如缺陷现象、测试步骤及相关日志等自动关联；
- 2) 应支持依据预设的规则库（如常见缺陷与根因的映射关系）进行简单匹配，输出大概率的根因候选结果；
- 3) 应支持检测到缺陷后自动触发根因分析流程。

2 级：

- 1) 应支持主动关联测试全流程数据，包括案例设计细节、测试执行日志、环境配置信息、代码提交记录等，构建完整的缺陷关联数据链，为根因分析提供全面的数据支撑；
- 2) 应引入简单的机器学习算法（如分类算法、关联规则挖掘），能够对缺陷现象进行深度拆解，结合上下文信息排除干扰因素，精准定位根因；
- 3) 应支持基于根因分析结果识别同类缺陷的潜在风险点，提出针对性的测试优化建议（并更新自身的根因分析规则库）。

3 级：

- 1) 应支持构建覆盖测试对象全生命周期的知识图谱，整合产品架构、代码逻辑、测试流程、历史缺陷数据、研发过程数据等多维度信息；
- 2) 应支持能够基于知识图谱实现缺陷根因的深度挖掘与关联推理；
- 3) 应支持在缺陷发生前通过对测试数据的趋势分析，预测可能出现的缺陷类型及根因；
- 4) 对于复杂的跨模块、跨系统缺陷（如分布式系统中的链路调用故障、微服务架构下的依赖冲突），能够通过知识图谱的关联推理，梳理清楚各模块、各环节的依赖关系，精准定位到底层的根因（如代码逻辑漏洞、架构设计缺陷、分布式事务处理异常等），并提供详细的根因溯源链路图；
- 5) 宜支持根据根因分析结果自动触发修复建议推送、相关测试案例更新、测试环境优化等操作，实现缺陷根因分析与测试流程优化的闭环管理；
- 6) 宜支持对已解决的缺陷进行系统性复盘，总结根因分析的经验教训，更新知识图谱与分析模型，实现自我演进。

7.3.3. 研发问题智能分析

1 级：

- 1) 应支持测试失败日志的基础信息提取与归档分类；
- 2) 应支持失败截图与日志的关联展示；
- 3) 应支持提供基础的错误信息搜索与匹配功能。

2 级：

- 1) 应支持失败原因的多维度分析，如代码、环境、数据、脚本等；
- 2) 应支持提供相似历史问题的智能推荐与解决方案关联；
- 3) 应支持问题严重程度的自动评估与优先级排序；
- 4) 应支持问题根因的初步推断分析，缩小排查范围。

3 级：

- 1) 应支持基于代码变更的影响面分析，快速定位变更风险；
- 2) 应支持问题修复建议的自动生成，提供代码级修改方案。

7.3.4. 智能缺陷预测

1 级：

- 1) 应支持统计分析历史缺陷数据（模块缺陷数、密度等）；
- 2) 应支持以图表形式展示缺陷趋势，辅助人工判断；
- 3) 应支持识别近期代码变更频繁的模块或文件；
- 4) 应支持静态代码质量指标与历史缺陷数据基础关联，给出高风险缺陷清单。

2 级：

- 1) 应支持整合多维度特征（代码复杂度、测试覆盖率等）；
- 2) 应支持构建多因子缺陷预测模型进行缺陷预测（如精准预测代码提交或改动引起的缺陷）；
- 3) 应支持根据预测的高风险缺陷自动生成测试任务建议。

3 级：

- 1) 应支持在持续集成流水线中实时缺陷预测；
- 2) 应支持预测缺陷类别（性能、安全等）及严重等级；
- 3) 应支持高风险代码提交后及时预警并触发验证流程；
- 4) 应支持多因子缺陷预测模型基于新数据自动反馈学习，形成自治闭环；
- 5) 宜支持构建细粒度预测模型，不仅能预测缺陷可能性，还能预测缺陷的潜在类型（如业务逻辑、安全漏洞）和严重等级。

8. 智能体工程能力（认证可选模块）

8.1. 自然语言理解

1 级：

- 1) 应支持测试需求文本的基础语义识别；
- 2) 应支持测试术语与关键字的准确提取；
- 3) 应提供执行/查询/生成等指令的意图识别。

2 级：

- 1) 应支持业务场景的语义解析与要素提取, 如登录/下单/搜索等；
- 2) 应支持测试需求的隐含条件推理；
- 3) 应支持维持多轮对话的上下文记忆，确保测试任务指令的连贯性；
- 4) 应支持需求歧义识别与澄清问题生成；
- 5) 应支持领域知识的语义扩展与同义词识别；
- 6) 应支持需求文档的结构化信息抽取。

3 级：

- 1) 应支持跨文档的需求关联分析与一致性检查；
- 2) 应具备领域自适应能力，能基于项目上下文动态调整语义理解模型；
- 3) 应支持基于业务规则的需求完整性智能校验；
- 4) 宜支持需求质量评估, 识别模糊、矛盾或缺失信息；
- 5) 宜支持增强语义推理能力。

8.2. DOM 智能分析

1 级：

- 1) 应支持基础页面元素的识别与定位；
- 2) 应支持解析标准 HTML 元素的属性与结构；
- 3) 应具有基础的选择器定位能力。

2 级：

- 1) 应支持动态页面内容的识别与适应；
- 2) 应支持分析复杂组件结构与交互关系；
- 3) 应支持可视化界面元素的语义理解；

- 4) 应支持元素状态变化的实时监测;
- 5) 应支持跨框架的组件识别与解析。

3 级:

- 1) 应支持页面设计的分析与优化建议;
- 2) 应支持预测用户交互路径与操作序列;
- 3) 宜支持基于计算机视觉的辅助元素识别;

8.3. 上下文工程

1 级:

- 1) 应支持当前测试任务的基础上下文记忆;
- 2) 应支持识别测试环境的基础配置信息;
- 3) 应支持简单的会话状态维护。

2 级:

- 1) 应支持跨任务的多维度上下文关联;
- 2) 应支持感知测试执行的环境变化;
- 3) 应支持对话记忆的持久化配置, 支持在长周期会话中维持任务上下文的一致性;
- 4) 应提供上下文驱动的策略调整;
- 5) 应支持多用户协作的上下文共享。

3 级:

- 1) 应支持测试上下文的预测与预加载;
- 2) 应支持构建测试知识图谱支持决策推理;
- 3) 宜支持自适应的上下文学习与优化。

8.4. 测试数据服务

1 级:

- 1) 应支持基础测试数据的生成与管理;
- 2) 应支持标准数据格式的解析与验证;
- 3) 应支持简单的数据模拟(测试数据生成)服务。

2 级:

- 1) 应支持复杂业务数据的生成;
- 2) 应支持数据依赖关系的自动分析;

- 3) 应支持测试数据的版本控制;
- 4) 应支持实现数据安全性与隐私保护;
- 5) 应支持多环境的数据一致性管理。

3 级:

- 1) 应支持测试任务的基础调度与执行;
- 2) 应支持预测测试数据需求并主动准备;
- 3) 宜支持数据质量的自动化评估与优化。

8.5. 元素定位

1 级:

- 1) 应支持识别常见 UI 元素类型 (输入框、按钮、下拉框、复选框等);
- 2) 应能在给定自然语言指令 (如 “点击登录按钮”) 后, 准确映射到对应界面区域;
- 3) 应支持在嵌套结构 (如 iframe、Shadow DOM、多窗口、原生弹窗) 中进行元素定位;
- 4) 应支持在定位失败时提供可读性提示 (如 “元素不存在” “未加载完成”);
- 5) 宜支持 Web、移动端等主流平台的基础定位。

2 级:

- 1) 应支持在界面存在轻微变化 (如文字调整、布局微调、主题切换、分辨率适配) 时, 保持目标元素的正确定位, 不因非关键属性变动导致定位失败;
- 2) 应避免依赖易变特征 (如动态 class、绝对坐标、XPath 索引), 提升定位鲁棒性;
- 3) 应支持结合上下文信息 (如页面状态、用户角色、操作流程) 进行歧义消解, 确保在多个相似元素中准确识别目标。

3 级:

- 1) 应具备在 UI 重构或版本迭代后自动恢复定位的能力, 在多次尝试内恢复准确操作;
- 2) 应支持基于历史执行数据动态优化定位策略;
- 3) 应支持在无结构化信息的场景 (如 Canvas、WebGL、原生 App) 下实现高精度定位 (准确率 $\geq 90\%$);
- 4) 应支持从失败案例中自动学习新模式;
- 5) 可支持跨平台统一定位表达。

8.6. API/SDK

1 级：

- 1) 应支持 API/SDK 的基本测试能力，包括接口调用、参数传递、响应验证等；
- 2) 应具备 API/SDK 的调用日志记录与基本错误提示能力，支持测试过程中能够追溯调用行为；
- 3) 应支持 API/SDK 的接口文档解析能力，能够自动提取接口定义、参数说明和返回格式，用于生成用例。

2 级：

- 1) 应支持对 API/SDK 的版本管理，能够识别不同版本的接口差异并进行测试适配；
- 2) 应支持对 API/SDK 的性能测试，包括响应时间、吞吐量等基本指标的监控；
- 3) 应支持对 API/SDK 的参数化测试，能够根据不同的测试场景动态调整参数；
- 4) 应支持对 API/SDK 的异常处理测试；
- 5) 应支持对 API/SDK 的集成测试，能够与测试工具链（如测试管理平台、缺陷管理系统、CI/CD 工具）进行深度集成，实现自动化测试流程；
- 6) 应支持对 API/SDK 的接口安全测试，能够检测潜在的安全漏洞（如越权访问、数据泄露等）。

3 级：

- 1) 应支持对 API/SDK 的智能回归测试，能够基于历史测试数据和变更记录，自动识别需要回归的接口并生成测试用例；
- 2) 应支持对 API/SDK 的智能场景化测试，结合历史调用记录和业务需求，生成组合场景测试，支持上下文参数传递；
- 3) 应支持对 API/SDK 的智能缺陷预测，能够基于历史缺陷数据和测试结果，预测 API/SDK 中可能存在的缺陷并提供修复建议；
- 4) 应支持对 API/SDK 的智能测试报告生成，能够自动生成详细的测试报告，包括测试覆盖率、性能指标、缺陷分析等内容，并支持可视化展示。

8.7. 测试管理

1 级：

- 1) 应支持测试任务的基础调度与执行；
- 2) 应支持测试结果的基础收集与存储；
- 3) 应支持简单的测试报告生成。

2 级:

- 1) 应支持测试流程编排;
- 2) 应支持测试质量的多维度分析;
- 3) 应支持测试资源的优化分配;
- 4) 应支持测试进度的预测;
- 5) 应支持测试风险的自动化识别。

3 级:

- 1) 应支持测试策略的自主优化与演进;
- 2) 应支持构建测试效能评估体系;
- 3) 宜支持基于大数据的测试决策分析。

8.8. DevOps

1 级:

- 1) 应支持与 CI/CD 工具的基础对接, 能监听代码仓库变更事件并自动触发测试任务;
- 2) 应支持测试结果的基础反馈至 CI/CD 流水线, 并生成基础的流水线执行报告;
- 3) 应支持测试环境的基础部署状态监控;
- 4) 应支持基础的任务容错能力 (自动重试/执行超时), 并记录关键操作日志。

2 级:

- 1) 应支持测试流程与 CI/CD 流水线的深度集成, 支持基于质量门禁的流水线控制;
- 2) 应支持自动化管理测试环境 (部署/回收/多环境配置同步) 与测试数据, 并监控测试资源使用情况;

- 3) 应支持提供关键运行时指标 (如任务队列长度、成功率、耗时) 的监控与告警。
- 4) 应支持高可用能力, 支持关键组件的冗余部署, 避免单点故障。

3 级:

- 1) 应支持基于质量风险的流水线调度、测试范围影响分析和测试部署策略优化;
- 2) 应支持跨团队的研发效能洞察 (提供从需求、开发、测试到部署的全链路效能度量和分析), 并能基于历史数据持续优化 DevOps 流程;
- 3) 应支持基于历史稳定性数据预测瓶颈, 并实现系统配置的自优化。

8.9. 可观测

1 级:

- 1) 应支持记录智能体单次任务的主要执行步骤、关键 prompt、模型版本、响应结果及耗时等关键活动信息。
- 2) 应支持采集测试流程基础状态（例：排队、执行、成功、失败），并支持可视化展示。
- 3) 应支持展示测试任务中被调用知识条目的名称、版本、调用时间等元信息。
- 4) 应支持对输入、输出与智能体行为等安全检测，包括敏感字段识别、脱敏校验与越权调用记录。

2 级：

- 1) 应支持任务级完整调用链的采集与可视化展示，并能按任务、产品、项目等不同维度进行数据聚合与统计分析。
- 2) 应支持测试全流程的端到端链路追踪，并可按模块/计划等维度展开。
- 3) 应支持展示多智能体之间的任务交接、消息传递与协作顺序，实现协作链路可观测。
- 4) 应支持对异常事件进行跨步骤、跨组件的关联分析，展示异常发生时的上下文、模型版本、依赖服务状态等信息。
- 5) 应支持构建知识条目的引用链路，展示知识与任务、用例、脚本等生成内容之间的关联，实现知识使用的跨流程可追踪。
- 6) 应支持将输入/输出风险、工具调用、外部依赖访问进行关联分析，识别潜在越权链路、风险模式与高危主体。

3 级：

- 1) 应支持基于完整输入、调用链、模型与环境快照对异常进行精准定位，并提供一键复现能力，可还原当次执行环境与决策过程，用于验证问题成因与修复效果。
- 2) 应支持分析知识变更对生成内容、执行结果与测试质量的影响，用于识别知识更新带来的风险、冲突或质量波动。
- 3) 应支持基于多维观测数据（效率、人效、覆盖率、模型表现等）进行综合化分析，形成任务级、产品级、团队级的效能洞察，为流程优化与资源配置提供决策依据。
- 4) 应支持关联 Token 消耗、算力成本与业务价值产出，形成投入产出比（ROI）评估能力。
- 5) 应具备持续优化机制，支持根据诊断结论自动生成优化建议，并支持 A/B 测试与效果回溯；
- 6) 应支持安全事件的全链路回放与责任归因，自动分析影响范围并生成合规审计报告。

9. 智能体构建

9.1. 模型配置

1 级：

- 1) 应支持至少接入一种大语言模型或专用模型作为智能体的核心推理引擎；
- 2) 应支持基础的模型采样参数（如 temperature, top-p）配置能力，允许用户或管理员完成配置；
- 3) 应支持模型 API 密钥、端点等连接配置信息的安全存储与管理；
- 4) 应支持对模型服务连通性与响应延迟的基础监控与告警；
- 5) 应支持模型注册，建立模型注册清单，记录模型名称、用途、版本及输入输出参数；
- 6) 应支持模型加载、卸载与健康检测；
- 7) 应支持模型调用接口，具备调用日志与错误记录。

2 级：

- 1) 应支持多模型路由与负载均衡策略，可根据任务类型、成本或性能要求智能选择最优模型，实现多模型的协作编排；
- 2) 应支持针对不同的测试任务场景（如脚本生成、结果分析）预置和调用经过精调（Fine-tuning）的领域专用模型；
- 3) 应支持对话记忆的持久化配置，支持在长周期会话中维持任务上下文的一致性；
- 4) 应支持模型性能与效果的对比评估功能，支持通过历史任务数据对多个模型进行 A/B 测试；
- 5) 应支持多模型版本共存与灰度、回滚机制；
- 6) 应支持不同推理后端（本地/远程、CPU/GPU 等）适配；
- 7) 应设定并监控模型服务 SLA；
- 8) 宜具备模型访问控制与调用审计能力。

3 级：

- 1) 应支持基于任务执行效果的反馈闭环，能自动优化和调整模型的选择策略与调用参数；
- 2) 应支持建立完善的成本、性能、质量监控体系，提供模型调用频次、token 消耗等使用度量的监控与成本分析；
- 3) 应支持在线学习或增量学习机制，能使模型利用测试过程中新产生的领域数据进行持续的适应性优化；
- 4) 宜具备模型表现衰退的自动检测与预警能力，并能触发模型的自动重训练或切换流程。

9.2. 提示词

1 级：

- 1) 应支持提示词的手动编辑与保存，支持提示词调用记录及执行结果摘要存储能力；
- 2) 应提供一套经过验证的、适用于基础测试场景提示词的内置模板库；
- 3) 应支持测试场景的基础提示词模板库配置；
- 4) 应具备动态内容填充功能，支持在提示词模板中插入变量（如需求描述、测试数据）；
- 5) 应支持基于角色的提示词配置，可为智能体赋予不同的测试角色（如安全测试专家、性能测试专家）。

2 级：

- 1) 应支持提示词版本的分类和管理，支持查看和回退到历史版本；
- 2) 应支持基于任务类型的提示词智能推荐；
- 3) 应支持提示词与模型、流程节点的绑定与追溯；
- 4) 应支持对提示词进行基础的安全与合规性检查，识别并过滤明显的敏感词或不安全指令；
- 5) 应支持提示词链的编排，能将复杂的测试任务分解为多步提示词流程并自动执行；

3 级：

- 1) 应支持提示词的参数化功能，支持在模板中嵌入变量，并根据具体场景自动填充；
- 2) 应支持根据测试资产（如需求文档、接口定义）的上下文，自动检索并填充到提示词中，宜具备多语境提示词功能；
- 3) 应支持多轮对话中提示词的上下文记忆与动态演进；
- 4) 宜支持多模态提示词工程，能协同处理与生成结合了文本、代码、截图等不同元素的复杂指令；
- 5) 宜具有提示词知识库，和经过验证的高效提示词模式，支持智能检索与推荐。

9.3. 流程编排

1 级：

- 1) 应支持将测试任务分解为节点，支持顺序、并行与重试；
- 2) 应支持常见测试工具或平台集成服务；
- 3) 应支持提供可视化流程编排界面，提供基础的条件分支配置、异常处理与重试机制等能力；
- 4) 应支持编排流程的版本管理、导入/导出功能；
- 5) 应支持手动或定时触发已编排好的测试流程。

2 级：

- 1) 应支持跨工具、跨环境的参数映射与适配；
- 2) 应支持条件分支、补偿机制与事件触发；

- 3) 应支持模板化流程复用、版本化管理与变更审计；
- 4) 应具备权限控制与失败隔离机制；
- 5) 应支持流程的实时运行状态监控与可视化，能够展示当前执行节点、输入输出数据及历史记录；
- 6) 应支持与外部工具（如测试管理平台、缺陷管理系统、CI/CD 工具）的深度集成，可在流程中调用其 API；
- 7) 应支持子流程的嵌套与调用，支持构建模块化、层次化的复杂测试工作流；
- 8) 应支持流程的并行执行编排，以提高测试效率。

3 级：

- 1) 应支持根据上下文动态调整，实现自适应编排；
- 2) 应具备流程的自我修复与自适应能力，当某个步骤因环境变更等原因失效时，能自动寻找替代方案并完成目标。
- 3) 应支持对流程的执行效率与质量进行持续监控与分析，并自动生成流程优化建议（如步骤合并、并行化改造）。
- 4) 宜支持多智能体协同编排；
- 5) 宜支持运行态回放与自动优化建议生成；
- 6) 可支持跨区域、跨集群的全局调度与弹性编排。

9.4. 效果验证

1 级：

- 1) 应支持智能体输出结果的基础正确性检查；
- 2) 应支持关键指标的阈值监控，如响应时延、正确率、通过率；
- 3) 应支持人工评审的结果标注与反馈记录；
- 4) 应支持记录智能体任务执行的原始输入、输出及上下文日志，以供回溯分析；
- 5) 应支持设定的质量阈值（如任务成功率>90%），并对不达标情况进行告警；
- 6) 应支持效果评估指标的自定义，允许团队根据自身业务目标设定特定的评估标准。

2 级：

- 1) 应支持建立自动化的效果评估体系，能通过规则引擎、模型自评或与其他工具结果比对等方式，对智能体输出进行自动化质量评估；
- 2) 应支持 AB 测试、灰度实验及质量门槛机制；
- 3) 应支持提供效果基线对比与回归检测机制；
- 4) 应支持边界场景与异常输入的鲁棒性验证；

- 5) 应支持评测结果与模型、提示词、流程的关联追溯;
- 6) 应支持多维度效果分析能力,可从任务类型、项目、时间周期等角度分析效果波动与趋势;
- 7) 应支持效果数据与业务数据(如缺陷逃逸率、测试效率提升)的关联分析,量化智能体的业务价值;
- 8) 宜支持验证结果的可视化分析与趋势跟踪。

3 级:

- 1) 应支持基于真实生产数据的在线效果评估;
- 2) 应支持预测智能体在新场景或新任务上的表现,并评估其泛化能力与潜在风险;
- 3) 应支持提供效果的可解释性报告,不仅能给出评估结果,还能清晰阐述得出该结论的依据与智能体的决策过程;
- 4) 宜支持效果验证标准的持续学习与自适应调整;
- 5) 宜支持安全、越权、对抗等专项评测与自动补救机制。

9.5. 知识库

1 级:

- 1) 应支持测试领域核心知识(如业务术语、测试方法、工具命令)的结构化存储与分类管理;
- 2) 应支持提供知识条目的手动增、删、改、查功能,并支持批量导入/导出;
- 3) 应支持知识条目的基础属性管理(如标题、内容、类型、标签、创建者、版本);
- 4) 应支持基于关键词的精确匹配和模糊搜索,能够快速检索到相关知识;

2 级:

- 1) 应支持从非结构化文档中自动抽取知识实体和关系,形成相应知识库;
- 2) 应支持对知识进行智能标签化与分类,支持基于语义的相似性检索和关联知识推荐;
- 3) 应支持知识的多版本管理与变更历史追溯,能够对比不同版本间的差异;
- 4) 应建立知识质量评估机制,支持对知识进行校验和标注;
- 5) 应支持知识库与测试工具链(如测试设计、执行、分析)的集成,实现知识在测试活动中的主动推送与应用;
- 6) 应支持生成知识库的使用情况报告,统计热点知识、知识缺口等;
- 7) 应支持知识图谱的构建,实现核心测试资产(如需求-用例-缺陷)的关联追溯。

3 级:

- 1) 应支持实现知识的自演进与自我优化,通过分析智能体与用户的交互反馈,自动发现新知

识、修正错误知识、淘汰过时知识；

- 2) 应支持基于知识推理的智能问答与决策支持，能够回答复杂业务问题并提供测试策略建议；
 - 3) 应具备知识发现能力，能从海量测试数据和生产数据中自动挖掘潜在的隐性知识；
 - 4) 宜支持跨领域知识的迁移与融合，能够引入外部知识（如行业最佳实践、新技术动态）并适配到本地上下文。
-